

Lab 07 — Token Standards on Hardhat + TrustKeys

Bài thực hành 07 — Chuẩn Token với Hardhat + TrustKeys

Time / Thời lượng: 50 minutes in class / 50 phút trên lớp. **Pass condition / Điều kiện đạt:** npx hardhat test all green **and** both contracts deployed to TrustKeys — import ClassToken in MetaMask and show a 100-CTK transfer, and decode a ClassBadge tokenURI (show the TA). / npx hardhat test xanh toàn bộ và đã deploy cả hai hợp đồng lên TrustKeys — nhập ClassToken vào MetaMask và chuyển 100 CTK, giải mã tokenURI của ClassBadge (cho trợ giảng xem).

Setup / Chuẩn bị (before class / trước buổi học):

```
node --version # must be >= 18 / phải >= 18
npm i @openzeppelin/contracts@5.0.2 # audited building blocks / thư viện đã kiểm toán
# MetaMask on TrustKeys: RPC https://lltestnet.trustkeys.network, chainId 11968
# Your dedicated TEST account must be funded by the instructor.
# Tài khoản THỬ riêng của bạn phải được giảng viên nạp tiền trước.
```

*Never paste a real mnemonic or a key that holds real funds anywhere in this lab. Use a **dedicated MetaMask test account** only. / Tuyệt đối không dán mnemonic thật hay khóa giữ tiền thật. Chỉ dùng tài khoản MetaMask thử riêng.*

*Why pin @openzeppelin/contracts@5.0.2? The newest OZ (5.6+) uses the Cancun mcopy opcode; TrustKeys' Geth targets **paris**, so pin 5.0.2 and keep the EVM version at paris. / OZ mới nhất dùng opcode Cancun mcopy; TrustKeys chạy EVM paris, nên ghim 5.0.2.*

Lab 7.0 — Project setup (5') / Khởi tạo dự án

```
mkdir tokens && cd tokens
npm init -y
npm i --save-dev hardhat @nomicfoundation/hardhat-toolbox dotenv
npm i @openzeppelin/contracts@5.0.2
npx hardhat init # choose "Create a JavaScript project"
```

In hardhat.config.js set solidity: { version: "0.8.24", settings: { optimizer: { enabled: true, runs: 200 } } } and add the trustkeys network (see solutions/hardhat.config.js). / Cấu hình compiler 0.8.24 + mạng trustkeys.

Q1 / Câu 1: Why do we build on OpenZeppelin instead of writing ERC-20 from scratch? / Vì sao dùng OpenZeppelin thay vì tự viết ERC-20?

Lab 7.1 — ClassToken (ERC-20 + tests) (15') / Viết token

Create contracts/ClassToken.sol. Requirements / Yêu cầu:

1. Inherit ERC20, ERC20Capped, ERC20Permit; name "ClassToken", symbol "CTK", cap $1,000,000 \times 10^{18}$. / Kế thừa 3 extension; cap 1.000.000.
2. In the constructor `_mint` the full cap to `initialHolder`. / Mint toàn bộ cap cho người nhận ban đầu.
3. Override `_update(...)` with `override(ERC20, ERC20Capped)` (both parents define it). / Override `_update` vì hai cha cùng định nghĩa.

Then write `test/ClassToken.test.js` with **at least**: - `name/symbol/decimals + cap() == totalSupply() == deployer balance`; - `transfer` moves balances and emits `Transfer`; - an `insufficient-balance` transfer reverts with `ERC20InsufficientBalance`; - `approve` → `transferFrom` by a spender; allowance decreases; - a **loop** over a list of amounts: recipient balance equals the running sum, `totalSupply()` stays constant (invariant). / vòng lặp bảo toàn cung.

`npx hardhat test`

Q2 / Câu 2: `decimals()` returns 18. If Alice's balance is 1000000000000000000, how many CTK is that, and where does the "18" actually live — on-chain or in the UI? / Số dư đó là bao nhiêu CTK, và số "18" nằm ở đâu? **Q3 / Câu 3:** Why does a DEX need `approve + transferFrom` instead of a plain transfer? What is the risk of `approve(spender, 2256-1)`? / Vì sao DEX cần hai bước? Rủi ro của infinite approval?

Lab 7.2 — ClassBadge (ERC-721, on-chain metadata) (15') / Viết NFT

Create contracts/ClassBadge.sol. Requirements / Yêu cầu:

1. Inherit ERC721 + Ownable; `mint(address to, string studentName)` is **onlyOwner**, uses `_safeMint`, returns the new `tokenId`. / `mint` chỉ owner, dùng `_safeMint`.
2. `tokenURI(tokenId)` returns a **data:application/json;base64,...** URI built **on-chain** — the JSON has name, description, an attributes array (a Student trait = the name), and an image field that is itself a base64 SVG data-URI. Use OZ Base64 + Strings. / metadata dựng on-chain, không IPFS.
3. Revert `EmptyName` on empty input; unminted id → `ERC721NonexistentToken` (from `_requireOwned`). / lỗi tên rỗng / token chưa tồn tại.

Then write `test/ClassBadge.test.js` that **decodes the base64** in JS and asserts the JSON fields:

```
const uri = await badge.tokenURI(0);
const json = Buffer.from(uri.split(",")[1], "base64").toString("utf8");
const meta = JSON.parse(json);
expect(meta.attributes.find(a => a.trait_type === "Student").value).toEqual("...");
```

Q4 / Câu 4: Where do the JSON and the image live? What did we avoid by NOT using IPFS on TrustKeys? / JSON và ảnh sống ở đâu? Ta tránh được điều gì khi không dùng IPFS? **Q5 / Câu 5:** What does the "safe" in `_safeMint` / `safeTransferFrom` actually check, and why? / Chữ "safe" kiểm tra điều gì?

Lab 7.3 — Deploy & interact on TrustKeys (10') / Triển khai

```
cp .env.example .env # put your TEST-account PRIVATE_KEY in .env
npx hardhat run scripts/deploy-token.js --network trustkeys
npx hardhat run scripts/deploy-badge.js --network trustkeys
```

1. **Import CTK in MetaMask:** "Import tokens" → paste the ClassToken address. Then send 100 CTK to a classmate:

```
TOKEN=0xYourToken TO=0xClassmate \
npx hardhat run scripts/interact-token.js --network trustkeys
```

2. **Decode a badge** (no explorer, no IPFS):

```
REGISTRY=0xYourBadge TOKEN_ID=0 \
npx hardhat run scripts/read-badge.js --network trustkeys
```

The script base64-decodes `tokenURI(0)` and prints the JSON + SVG. / Script giải mã base64 và in JSON + SVG.

Q6 / Câu 6: TrustKeys has no public explorer. How did you confirm the 100-CTK transfer landed? / Không có explorer — bạn xác nhận giao dịch bằng cách nào?

Lab 7.4 — BONUS: gasless approval with permit (EIP-2612) / permit

Sign an approval **off-chain**, submit it from a **second** account:

```
const domain = { name: "ClassToken", version: "1", chainId,
                  verifyingContract: await token.getAddress() };
const types = { Permit: [ {name:"owner",type:"address"},
                           {name:"spender",type:"address"},
                           {name:"value",type:"uint256"},{name:"nonce",type:"uint256"},
                           {name:"deadline",type:"uint256"} ] };
const nonce = await token.nonces(owner.address);
const sig = await owner.signTypedData(domain, types,
  { owner: owner.address, spender: spender.address, value,
    nonce, deadline });
const { v, r, s } = ethers.Signature.from(sig);
await token.connect(relayer).permit(owner.address, spender.address, value,
  deadline, v, r, s);
```

Assert the allowance is set and nonces(owner) incremented; add a test that an **expired deadline** reverts ERC2612ExpiredSignature. / Kiểm tra allowance + nonce tăng; deadline hết hạn phải revert.

ethers v6 (hardhat-toolbox v5) uses signer.signTypedData(...); in ethers v5 it was wallet._signTypedData(...).

Q7 / Câu 7: What stops someone from replaying your permit signature on a second chain or a second time? / Điều gì chặn tấn công replay chữ ký permit?

Reference solution / Lời giải tham khảo

A full Hardhat project (contracts + 14 passing tests + deploy/interact scripts) is in lab/solutions/. Single-file Remix versions are lab/ClassToken_remix.sol and lab/ClassBadge_remix.sol for the Remix path (<https://remix.trustkeys.com>). / Lời giải đầy đủ ở lab/solutions/; bản Remix một tệp cho đường Remix.

Answers (short) / Đáp án ngắn: - **Q1** Audited, battle-tested, gas-reviewed; less surface for bugs. / Đã kiểm toán, ít lỗi. - **Q2** 1 CTK; the 18 is display-only metadata — on-chain everything is integer base units. / 1 CTK; "18" chỉ để hiển thị. - **Q3** A contract runs no code on a plain transfer, so it can't react; transferFrom lets it pull inside its own tx. Infinite approval = a hacked spender drains your whole balance. / transfer không chạy code người nhận; infinite approval bị hack là mất sạch. - **Q4** Both in the contract's own storage/bytecode; we avoided needing a pinning service. / Nằm trong hợp đồng; tránh phải pin IPFS. - **Q5** It checks the recipient contract implements onERC721Received, so NFTs aren't stranded in contracts that can't move them. / Kiểm tra người nhận là hợp đồng có onERC721Received. - **Q6** ethers view calls / balanceOf (and event logs) — same as MetaMask. / Bằng lời gọi view ethers + MetaMask. - **Q7** The signature is bound to chainId, the contract address, a per-owner nonce, and a deadline. / Chữ ký gắn chainId + địa chỉ hợp đồng + nonce + deadline.